

A COMPLETE PRACTITIONER'S GUIDE

Advanced Retrieval- Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE NINE

RAG Evaluation through RAGAS

Beginner to Industry Level • Assumes only basic Python

MODULE NINE

RAG Evaluation through RAGAS

Every earlier module ended with the same instruction: "measure it." This module finally makes measurement rigorous. You cannot improve a RAG system you cannot score, and "it looks good" is not a score. You will learn the RAGAS framework, the four core metrics that quantify RAG quality — faithfulness, answer relevancy, context precision, and context recall — and, crucially, how each metric localises a failure to either retrieval or generation. You will implement metrics from scratch to understand their mechanism, run the `evaluate()` API, build an automated evaluation pipeline, and learn benchmarking strategies that turn RAG development from guesswork into engineering.

WHERE WE ARE

Modules 1–8 built RAG: indexing, retrieval, advanced patterns, agentic systems. At every turn the advice was "tune by measurement" — but the measurement itself was deferred to here. This module supplies it. RAG evaluation is not a final polish step; it is the instrument panel you use *throughout* development to decide whether a change helped. Module 10 will then put a RAGAS evaluation suite into the production capstone.

9.1 Introduction

Here is an uncomfortable truth about everything you have built so far. You have made dozens of decisions — chunk size, overlap, embedding model, `k`, MMR versus hybrid search, whether to add re-ranking, RAG Fusion, or a corrective loop. And for every one of those decisions, the guidance was the same: *measure it, then decide*. Yet you have not actually measured anything. You have judged your RAG systems the way almost everyone judges their first RAG system: by typing in a few questions, glancing at the answers, and concluding "looks good."

"Looks good" is not engineering. It is the single biggest reason RAG projects stall. Consider what eyeballing a few answers cannot tell you:

- Did the answer happen to be right because retrieval was good — or because the model already knew it and ignored the retrieved context entirely?
- When you changed the chunk size last week, did quality improve, degrade, or stay flat? You have no number from before to compare against.

- Your system answers your five favourite test questions perfectly. How does it do on the *hundred* questions you have not thought of?
- A user reports a bad answer. Was the cause bad retrieval, or bad generation? Without measurement you are guessing.

RAG evaluation is the discipline that replaces "looks good" with numbers. And it is harder than it sounds, because a RAG answer is open-ended free text — there is no single correct string to compare against, the way there is for a classification model. You cannot just check `answer == expected`.

The breakthrough that makes RAG evaluation practical is twofold. First, **decompose** the quality of a RAG answer into a small set of distinct, measurable dimensions. Second, use an **LLM as the judge** to score each dimension — because judging whether a paragraph is faithful to a source is itself a language task that LLMs do well. **RAGAS** (Retrieval-Augmented Generation Assessment) is the standard framework that packages exactly this approach.

This module teaches you to evaluate RAG properly: what RAGAS is, the four core metrics and what each one isolates, how to implement them and run the `evaluate()` API, and how to build automated evaluation into your development loop.

9.2 Why This Topic Matters

WHY THIS MATTERS

There is an old engineering maxim: *you cannot improve what you cannot measure*. RAG is its perfect illustration. Without evaluation, every change to your system is a coin flip — you tweak chunk size, the answers *feel* different, and you have no idea whether you helped or hurt. With evaluation, every change becomes an experiment with a result. Evaluation is what turns RAG development from anxious tinkering into systematic engineering. It is also non-negotiable for production: you cannot responsibly deploy a system whose quality you have never quantified, and you cannot detect when a deployed system silently degrades.

Four concrete reasons this module repays close study:

1. **It validates every prior module.** Chunk size, embedding model, retriever choice, advanced patterns — the only honest way to know if any of them helped is to measure. This module is the proof step for all the rest.
2. **It localises failures.** The single most valuable thing evaluation gives you is the answer to "*where did it go wrong — retrieval or generation?*" The metrics pinpoint the broken stage so you fix the right thing.
3. **It enables regression detection.** A deployed RAG system can degrade silently — a data update, a model change, a drifting corpus. Evaluation is the alarm system.

4. **It is a core professional skill.** "How do you evaluate a RAG system?" and "What is faithfulness?" are standard senior interview questions. Being able to talk metrics fluently signals that you have done this for real.

9.3 A Beginner-Friendly Explanation

Forget frameworks for a moment. Imagine you are a teacher grading a student's open-book exam answer. The naive way to grade is to read the final answer and mark it right or wrong. But that tells you almost nothing about *how to help the student improve*. A student who wrote a correct answer might have reasoned brilliantly — or copied the wrong page and gotten lucky. A student who wrote a wrong answer might have had the right pages but misread them, or had the wrong pages entirely.

A good teacher grades the whole **process**, asking four separate questions:

1. *Did the student turn to the right pages of the book?* (Were the pages they used actually relevant to the question?)
2. *Did the student turn to ALL the pages they needed?* (Or did they miss a page that contained part of the answer?)
3. *Does the student's answer actually stick to what those pages say* — or did they make things up that the pages do not support?
4. *Does the answer actually address the question that was asked* — or is it on a tangent, however true?

These four questions are exactly the four RAGAS metrics. Question 1 is **context precision**. Question 2 is **context recall**. Question 3 is **faithfulness**. Question 4 is **answer relevancy**.

Notice how powerful this decomposition is. If a student scores badly on questions 1 and 2, the problem is *finding the right pages* — in RAG terms, the **retriever** is broken. If they score badly on question 3, the problem is *what they did with the pages* — the **generator** invented things. If they score badly on question 4, the generator went off-topic. By grading four dimensions instead of one, you do not just know *that* the answer was bad — you know *which part of the system to fix*.

That is the entire idea of RAG evaluation: do not grade the final answer alone; grade the four dimensions of the process, because each dimension points at a specific, fixable component.

DEFINITION

RAG evaluation is the practice of quantitatively measuring the quality of a RAG system by scoring several distinct dimensions of its behaviour — rather than only judging the final answer. **RAGAS** (Retrieval-Augmented Generation Assessment) is an open-source framework that computes these dimension scores automatically, using an LLM as the judge.

9.4 Real-World Analogy: The Examiner Who Grades the Working

REAL-WORLD ANALOGY

A weak maths examiner reads only the final number in the box and marks it right or wrong. A strong examiner grades the **working** — every line of the solution — because the working reveals *where* a mistake happened and therefore what the student must practise.

RAGAS is the strong examiner for a RAG system. It does not just look at the final answer. It inspects the working: which documents were retrieved, whether they were the right ones, whether all the needed ones were found, whether the answer was actually built from those documents or partly invented, and whether the answer addressed the real question. Two RAG systems can produce the same correct answer to your favourite test question; the examiner who grades the working will tell you that one did it by sound retrieval and faithful generation, and the other got lucky — and only the first will keep working on the questions you have not tested yet.

The analogy carries the module's central message: **grade the process, not just the product**. A RAG answer is the product; retrieval quality and generation quality are the process. Measuring the process is what makes the system improvable.

9.5 Core Concepts: The Four Data Elements and Two Halves

9.5.1 The Four Data Elements

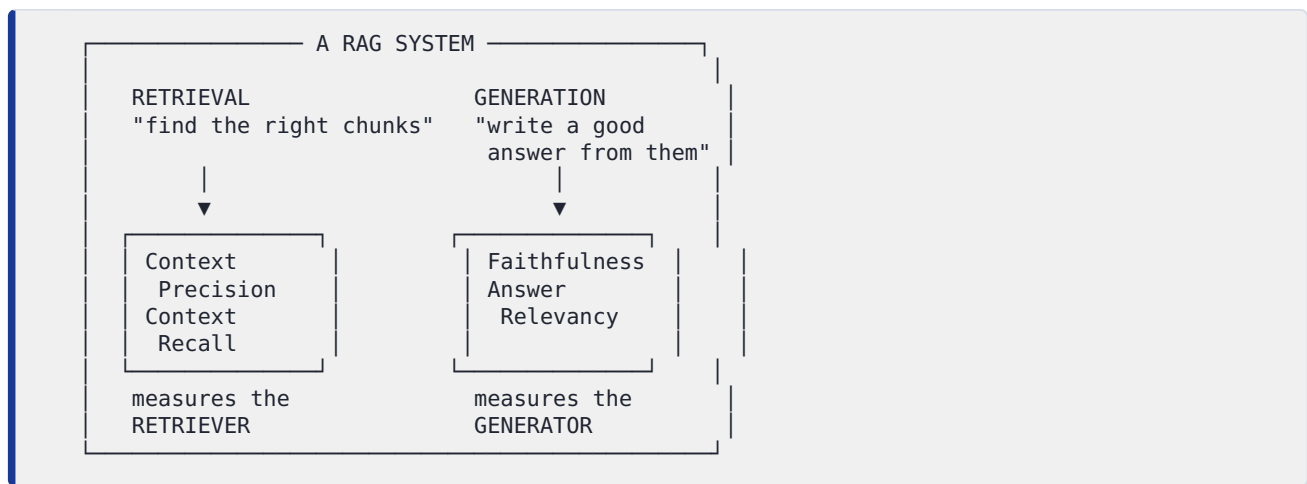
Every RAGAS metric is computed from some combination of **four pieces of data** per test question. Learn these four; everything else is built on them.

- **question** — the user's query. (In current RAGAS field naming, `user_input`.)
- **answer** — the answer your RAG system actually generated. (`response`.)
- **contexts** — the chunks your retriever actually returned for that question. (`retrieved_contexts`.)
- **ground_truth** — the *ideal*, human-written reference answer for that question. (`reference`.)

The first three are produced automatically by *running your RAG system* on a test question. The fourth — `ground_truth` — must be **created by a human** (or carefully curated), because it is the gold standard the system is judged against. Some metrics need `ground_truth`; some do not. Knowing which is which is essential, and we will flag it for every metric.

9.5.2 The Two Halves of a RAG System

Recall from Module 1 that RAG has two stages that can each fail independently: **retrieval** (finding the right chunks) and **generation** (writing a good answer from them). RAGAS's metrics split cleanly along exactly this line:



This split is the most important idea in the module. **Context precision** and **context recall** measure the *retriever*. **Faithfulness** and **answer relevancy** measure the *generator*. When a metric scores low, you immediately know which half of your system to investigate. Hold this two-by-two firmly; the rest of the module fills it in.

9.6 What Is RAGAS

RAGAS is an open-source Python framework purpose-built for evaluating RAG pipelines. Its core idea is **LLM-as-judge** evaluation: because judging qualities like "is this answer faithful to this source?" is itself a language-understanding task, RAGAS uses an LLM (and an embedding model) under the hood to perform the grading automatically.

This LLM-as-judge approach is what makes RAGAS practical, and you should understand both its strength and its caveat:

- **The strength:** it scales. A human evaluating 200 test questions across four dimensions is days of tedious work. RAGAS does it in minutes, automatically and repeatedly, so you can re-evaluate after every change.
- **The caveat:** the judge is itself an LLM, so the scores are *estimates*, not perfect ground truth. They carry some noise and depend on the quality of the judge model. The right

way to use RAGAS scores is as **reliable relative signals** — "version B scored higher than version A" is trustworthy and actionable — rather than as absolute, decimal-perfect verdicts.

In this book, RAGAS is configured to use **Gemini** as both its judge LLM and its embedding model, keeping the entire evaluation pipeline on the free tier.

COMMON MISTAKE

Treating a RAGAS score as an exact, absolute truth — agonising over whether faithfulness is "really" 0.87 or 0.84. Because the judge is an LLM, individual scores have noise; the same evaluation run twice may differ slightly. Use RAGAS the way it is meant to be used: for *comparison and tracking*. "Did this change move the score up or down, across a decent-sized test set?" is the question RAGAS answers reliably. "Is the score exactly 0.87?" is not.

9.7 Retrieval Evaluation vs Generation Evaluation

Before the metric details, internalise the diagnostic logic, because it is the practical pay-off of the whole module.

If retrieval metrics are low (context precision and/or context recall), the *retriever* is the problem. The fixes live in Modules 2–6: chunking strategy, chunk size and overlap, the embedding model, **k**, the retrieval technique, hybrid search, re-ranking. No improvement to the generator can fix a retrieval problem — the generator cannot use evidence it never received.

If retrieval metrics are healthy but generation metrics are low (faithfulness and/or answer relevancy), the *generator* is the problem. The fixes live in Modules 1 and 7: the prompt (especially the grounding instruction), the LLM choice, generation temperature, or a self-correcting pattern like Self-RAG.

This is why you measure all four metrics, not just one. A single overall "quality score" tells you *that* something is wrong; the four-metric breakdown tells you *what* — and *what* is the only thing you can act on.

9.8 The Four Core Metrics in Detail

For each metric: what it measures, which data it needs, how it is computed, what a low score means, and which half of the system it diagnoses. All four are scored from **0 to 1**, where **higher is better**.

9.8.1 Faithfulness — "Is the answer grounded in the evidence?"

Faithfulness measures whether the generated answer is **factually consistent with the retrieved context**. Every claim in the answer should be supported by the context the system was given. Faithfulness is the direct, quantitative measure of **hallucination**: a low faithfulness score means the answer contains claims the context does not back up.

- **Needs:** `answer` + `contexts`. (No `ground_truth` required.)
- **How it is computed:** the answer is broken into individual factual **claims**; each claim is checked against the retrieved context to see whether the context supports it; the score is the fraction of claims that are supported.

$$\text{faithfulness} = \frac{\text{number of answer claims supported by the context}}{\text{total number of claims in the answer}}$$

- **A low score means:** the generator is hallucinating — inventing or distorting information not present in the context. This is a **generation** problem (the prompt is too weak to ground the model, the temperature is too high, or the model is overriding context with parametric memory).

Faithfulness is, for most teams, the single most important RAG metric, because an unfaithful answer is a confidently-stated falsehood — the exact failure mode RAG exists to prevent.

9.8.2 Answer Relevancy — "Does the answer address the question?"

Answer relevancy measures whether the generated answer **actually addresses the user's question** — directly, completely, and without padding, evasion, or off-topic content.

- **Needs:** `question` + `answer` (and `contexts`).
- **How it is computed:** RAGAS uses the LLM to generate several questions *from the answer* — reverse-engineering "what question would this answer be a good response to?" — then measures the embedding similarity between those generated questions and the *original* question. If the answer truly addresses the original question, the reverse-generated questions will be very similar to it.
- **A low score means:** the answer is evasive, incomplete, or off-topic. This is a **generation** problem.

A subtle but critical point: **answer relevancy does not measure factual correctness**. An answer can be perfectly *relevant* (it addresses the question head-on) while being *wrong* (it states a false fact). Relevancy asks "is this on-topic and complete?"; faithfulness asks "is this true to the evidence?" They are different questions, which is exactly why you need both.

COMMON MISTAKE

Assuming a high answer-relevancy score means the answer is correct. It does not. Relevancy measures whether the answer is *on-topic and addresses the question*; it says nothing about whether the facts in it are true. A confident, on-topic, completely wrong answer can score high on relevancy. **Faithfulness** is the metric that checks truthfulness against the evidence. Always read relevancy and faithfulness together — a high-relevancy, low-faithfulness answer is the dangerous case: fluent, on-topic, and hallucinated.

9.8.3 Context Precision — "Is the retrieval high signal, low noise?"

Context precision measures the **signal-to-noise ratio of the retrieved context**: of the chunks the retriever returned, how many are actually relevant — and are the relevant ones ranked *near the top*? It rewards a retriever that returns relevant chunks and ranks them highly, and penalises one that pads the results with irrelevant chunks or buries the good ones low.

- **Needs:** `question` + `contexts` + `ground_truth` (or a reference for what is relevant).
- **How it is computed:** for each retrieved chunk, RAGAS judges whether it is relevant to answering the question, then computes a rank-aware score that is higher when relevant chunks appear at the top of the retrieved list.
- **A low score means:** the retriever is returning noise — irrelevant chunks — or ranking the genuinely relevant ones too low. This is a **retrieval** problem (revisit `k`, the retrieval technique, re-ranking from Module 6, or chunking).

9.8.4 Context Recall — "Did retrieval find everything needed?"

Context recall measures whether the retriever fetched **all** the information required to answer the question. Where context *precision* asks "is what I retrieved clean?", context *recall* asks "is what I retrieved *complete*?" — did the retriever miss a chunk that contained part of the answer?

- **Needs:** `question` + `contexts` + `ground_truth`. (Recall *requires* the ground-truth answer, because it must know what the complete answer contains.)
- **How it is computed:** the `ground_truth` answer is broken into claims; for each claim, RAGAS checks whether that claim *can be attributed to* the retrieved context. The score is the fraction of ground-truth claims that the retrieved context actually covers.

$$\text{context recall} = \frac{\text{ground-truth claims found in the retrieved context}}{\text{total claims in the ground-truth answer}}$$

- **A low score means:** the retriever missed necessary information — the answer was *in the knowledge base* but the retriever did not bring back the chunk containing it. This

is a **retrieval** problem, and a serious one: if a needed chunk was never retrieved, the generator literally cannot produce a complete answer, no matter how good it is.

9.8.5 The Four Metrics at a Glance

Metric	Question it answers	Diagnoses	Needs ground truth?	Low score means
Faithfulness	Is the answer grounded in the retrieved context?	Generation	No	The model is hallucinating
Answer Relevancy	Does the answer address the question?	Generation	No	The answer is evasive / off-topic
Context Precision	Is the retrieved context relevant and well-ranked?	Retrieval	Yes (typically)	The retriever returns noise / poor ranking
Context Recall	Did retrieval find all needed information?	Retrieval	Yes	The retriever missed needed chunks

INTERVIEW INSIGHT

"What are the main RAG evaluation metrics?" expects all four, each in one sentence, *grouped* by what they diagnose. Retrieval metrics: **context precision** (are the retrieved chunks relevant and well-ranked — signal vs noise) and **context recall** (did retrieval find everything needed — completeness). Generation metrics: **faithfulness** (is the answer grounded in the context — the hallucination measure) and **answer relevancy** (does the answer actually address the question). The point that signals depth: each metric *localises the failure* to retrieval or generation, so the four-metric breakdown tells you not just that quality is low but which component to fix.

9.9 Diagnosing a System by Reading the Metrics Together

The metrics are most powerful read as a *pattern*. Here is how common combinations diagnose a system:

- **Low context recall.** The retriever is missing needed chunks. Nothing downstream can compensate. Fix retrieval first — chunking, embedding model, , retrieval technique. *This is the highest-priority failure:* a needed chunk that was never retrieved makes a complete answer impossible.
- **Good recall, low context precision.** The retriever finds the needed chunks but also drags in noise or ranks badly. Add re-ranking (Module 6), lower , or improve the retrieval technique.

- **Good retrieval metrics, low faithfulness.** The right context was provided, but the model is inventing claims anyway. Fix generation: strengthen the grounding instruction in the prompt, lower the temperature, or add a Self-RAG faithfulness check.
- **Good retrieval and faithfulness, low answer relevancy.** The answer is grounded and true but does not squarely address the question — evasive, incomplete, or rambling. Fix the prompt to demand a direct, complete answer.
- **All four healthy.** The system is genuinely working — and now you have the numbers to *prove* it, and a baseline to defend future changes against.

WHY THIS MATTERS

This diagnostic table is the real product of the module. Before evaluation, a bad answer is a mystery and you fix things at random. After evaluation, a bad answer comes with a fingerprint — a pattern of four scores that points straight at the broken component. RAG debugging stops being guesswork and becomes a decision procedure: read the four numbers, identify the failing stage, apply the matching fix from the relevant module, re-measure. That loop is what separates a professional RAG engineer from someone who tinkers and hopes.

9.10 The evaluate() API

RAGAS packages all of this into one primary function: `evaluate()`. You give it a dataset of test samples and a list of metrics; it runs every metric over every sample using the judge LLM and returns the scores.

The workflow is three steps:

1. **Build an evaluation dataset.** For each test question, you need the four data elements. The `question` and `ground_truth` you author by hand (the test set). The `answer` and `contexts` you produce by *running your RAG system* on each question.
2. **Choose metrics** — typically the four core metrics.
3. **Call `evaluate()`** — passing the dataset, the metrics, and the configured judge LLM and embedding model.

```

from ragas import evaluate, EvaluationDataset
from ragas.metrics import (
    Faithfulness, ResponseRelevancy,
    LLMContextPrecisionWithReference, LLMContextRecall,
)

scores = evaluate(
    dataset=evaluation_dataset,          # the four data elements per sample
    metrics=[Faithfulness(), ResponseRelevancy(),
             LLMContextPrecisionWithReference(), LLMContextRecall()],
    llm=judge_llm,                      # Gemini, wrapped as the judge
    embeddings=judge_embeddings,        # Gemini embeddings for the judge
)
print(scores)  # → average score per metric across the dataset

```

A NOTE ON RAGAS VERSIONS

RAGAS is an actively evolving framework, and its exact import paths and class names change between versions — the metric named *answer relevancy* has appeared as both `answer_relevancy` and `ResponseRelevancy`; dataset construction has used both a plain dictionary form and an `EvaluationDataset` class. The four *concepts* in this module are stable; the *API surface* is not. Always check the current RAGAS documentation for the exact import names for your installed version, and treat the code here as the correct shape rather than a frozen incantation.

9.11 Automated Evaluation Pipelines

A one-off evaluation is useful; an *automated* evaluation pipeline is transformative. The goal is to make evaluation a routine, repeatable step you run after every meaningful change — not a special event.

An automated evaluation pipeline has these parts:

- **A fixed test set.** A curated, version-controlled set of representative questions with hand-written `ground_truth` answers. It must stay fixed, so scores are comparable across runs. (Curating it well is the hard part — see best practices.)
- **A run-and-score function.** Code that takes a RAG system, runs it over the whole test set to collect `answer` and `contexts`, and calls `evaluate()` to produce the four metric scores.
- **A record of results.** Each evaluation run's scores are logged with a label (which configuration produced them), so you can compare version A against version B against version C over time.
- **A trigger.** The pipeline runs whenever it matters — before merging a change, on a schedule, or as a continuous-integration check (Module 10 connects this to CI/CD).

With this in place, the development loop becomes disciplined: *change one thing → run the evaluation pipeline → compare the scores → keep the change only if the numbers improved*. Every modification from Modules 2–8 — a new chunk size, a different

embedding model, adding re-ranking — becomes a measured experiment rather than a hopeful guess.

BEST PRACTICE

Change **one variable at a time** between evaluation runs. If you simultaneously change the chunk size, swap the embedding model, and add re-ranking, then re-evaluate, a score change tells you nothing — you cannot attribute it to any one change, and a helpful change and a harmful one may have cancelled out. Disciplined evaluation is controlled experimentation: isolate the variable, measure its effect, decide, then move to the next. This is slower per step and far faster overall.

9.12 Benchmarking Strategies

Beyond a single test set, mature RAG teams adopt broader benchmarking practices:

- **A representative test set.** The test questions must mirror *real* usage — the easy questions and the hard ones, the well-phrased and the messy, the in-scope and the out-of-scope. A test set of only easy questions produces flattering, useless scores.
- **Stratified evaluation.** Group test questions by type — simple fact lookup, multi-hop, out-of-scope, conversational — and read the metrics *per group*. A system can score 0.9 overall while scoring 0.4 on multi-hop questions; the average hides the weakness, the breakdown reveals it.
- **Component ablation.** To measure a component's worth, evaluate the system *with* and *without* it. The score difference is that component's measured contribution — the rigorous way to decide whether re-ranking, RAG Fusion, or a corrective loop is *earning its cost*.
- **Baseline and regression tracking.** Establish the naive-RAG score as a baseline, and re-run the suite regularly so a silent degradation — from a data update, a model change, corpus drift — shows up as a dropped number.
- **Including a "should refuse" set.** Deliberately include questions whose answers are *not* in the knowledge base, and check that the system honestly declines rather than hallucinating. A system that scores well on answerable questions but invents answers to unanswerable ones is not production-ready.

9.13 Workflow

The evaluation-driven development loop, which should govern all of Modules 2–8:

1. **Build a test set** — representative questions with hand-written ground-truth answers, version-controlled and fixed.

2. **Run the current system** over the test set to collect each sample's `answer` and `contexts`.
3. **Score with `evaluate()`** — the four core metrics — to get a baseline.
4. **Read the four metrics** and diagnose: is the weak point retrieval (precision/recall) or generation (faithfulness/relevancy)?
5. **Make one targeted change** to the diagnosed component.
6. **Re-run the evaluation** and compare scores to the baseline.
7. **Keep the change only if the numbers improved**; then return to step 4 for the next-weakest dimension.

9.14 Step-by-Step Implementation and Code

We will do two things: implement a *simplified faithfulness metric from scratch* — so you understand the mechanism beneath the framework — and then run the real RAGAS `evaluate()` API. Save the following as `module9_evaluation.py`.

```

"""
module9_evaluation.py
RAG evaluation: a from-scratch faithfulness metric to show the mechanism,
then the real RAGAS evaluate() API with all four core metrics.
Stack: Python + LangChain + Gemini + RAGAS.
Install: pip install ragas langchain-google-genai datasets python-dotenv
"""

import os
from dotenv import load_dotenv
from langchain_google_genai import (
    ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings)
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", temperature=0)
embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

# ===== PART A: FAITHFULNESS FROM SCRATCH (the mechanism) =====

claim_extractor = (ChatPromptTemplate.from_template(
    "Break the ANSWER into a list of simple, standalone factual claims. "
    "Return one claim per line, nothing else.\n\nANSWER: {answer}")
    | llm | StrOutputParser())

claim_verifier = (ChatPromptTemplate.from_template(
    "Can the CLAIM be inferred from the CONTEXT? Reply only 'yes' or 'no'.\n"
    "CONTEXT: {context}\n\nCLAIM: {claim}")
    | llm | StrOutputParser())

def simple_faithfulness(answer: str, contexts: list[str]) -> float:
    """A simplified faithfulness score: supported claims / total claims."""
    context_text = "\n".join(contexts)
    claims = [c.strip() for c in
        claim_extractor.invoke({"answer": answer}).splitlines()
        if c.strip()]
    if not claims:
        return 0.0
    supported = 0
    for claim in claims:
        verdict = claim_verifier.invoke(
            {"context": context_text, "claim": claim}).strip().lower()
        if "yes" in verdict:
            supported += 1
    print(f" [faithfulness] {supported}/{len(claims)} claims supported")
    return supported / len(claims)

# ===== PART B: THE REAL RAGAS evaluate() API =====

def run_ragas_evaluation():
    """Evaluate a small RAG test set with the four core RAGAS metrics."""
    from ragas import evaluate, EvaluationDataset
    from ragas.metrics import (
        Faithfulness, ResponseRelevancy,
        LLMContextPrecisionWithReference, LLMContextRecall)
    from ragas.llms import LangchainLLMWrapper
    from ragas.embeddings import LangchainEmbeddingsWrapper

    # Wrap Gemini so RAGAS uses it as the judge model
    judge_llm = LangchainLLMWrapper(llm)
    judge_emb = LangchainEmbeddingsWrapper(embeddings)

    # An evaluation dataset: each sample has the four data elements.
    # In practice, 'response' and 'retrieved_contexts' come from RUNNING
    # your RAG system; 'user_input' and 'reference' are hand-authored.

```

```

samples = [
    {
        "user_input": "How many paid annual leave days do full-time "
                      "staff get?",
        "retrieved_contexts": [
            "Full-time staff receive 24 days of paid annual leave "
            "per calendar year, accrued monthly."],
        "response": "Full-time staff receive 24 days of paid annual "
                    "leave per year.",
        "reference": "Full-time employees are entitled to 24 days of "
                    "paid annual leave per calendar year.",
    },
    {
        "user_input": "How many sick days are provided?",
        "retrieved_contexts": [
            "The office observes all national public holidays."],
        "response": "Employees get 15 paid sick days each year.",
        "reference": "Sick leave covers up to 10 paid days per year.",
    },
]

dataset = EvaluationDataset.from_list(samples)
result = evaluate(
    dataset=dataset,
    metrics=[Faithfulness(), ResponseRelevancy(),
             LLMContextPrecisionWithReference(), LLMContextRecall()],
    llm=judge_llm,
    embeddings=judge_emb,
)
return result

if __name__ == "__main__":
    print("=== PART A: faithfulness from scratch ===")
    good = simple_faithfulness(
        "Full-time staff receive 24 days of paid leave.",
        ["Full-time staff receive 24 days of paid annual leave per year."])
    bad = simple_faithfulness(
        "Full-time staff receive 50 days of paid leave and a free car.",
        ["Full-time staff receive 24 days of paid annual leave per year."])
    print(f" grounded answer -> {good:.2f}")
    print(f" hallucinated one -> {bad:.2f}")

    print("\n=== PART B: RAGAS evaluate() ===")
    print(run_ragas_evaluation())

```

9.15 Line-by-Line Code Explanation

Setup. The Gemini chat model and embedding model are created once; `temperature=0` makes the judging deterministic, which matters — an evaluator should be as consistent as possible.

Part A — faithfulness from scratch. This part exists to demystify the metric: RAGAS is not magic, it is exactly this mechanism, refined. Two small LCEL chains do the work. `claim_extractor` prompts the LLM to break an answer into a list of standalone factual claims — this is the *decomposition* step. `claim_verifier` prompts the LLM to judge whether a single claim can be inferred from the context — a strict yes/no. `simple_faithfulness` ties them together: it joins the contexts, extracts the answer's claims, runs the verifier on each claim, counts how many are supported, and returns `supported /`

`total`. That ratio is faithfulness. Reading this function makes the metric concrete — claim extraction plus per-claim verification, scored as a fraction. RAGAS's real implementation is more sophisticated, but conceptually identical.

Part B — the real RAGAS API. `LangchainLLMWrapper` and `LangchainEmbeddingsWrapper` adapt the Gemini models into the form RAGAS expects, so RAGAS uses Gemini as its judge — keeping evaluation on the free tier. The `samples` list is the evaluation dataset, and each sample carries the **four data elements**: `user_input` (the question), `retrieved_contexts` (the chunks), `response` (the generated answer), and `reference` (the ground truth). The first sample is a healthy case — the answer matches a context that genuinely supports it. The **second sample is deliberately broken in two ways at once**: its retrieved context is about public holidays (irrelevant to a sick-leave question — this will tank *context precision* and *context recall*), and its answer claims "15 sick days" which neither the context nor the reference supports (this will tank *faithfulness*). `EvaluationDataset.from_list` builds the dataset; `evaluate()` runs all four metrics over both samples and returns the average score per metric. Running it, you will see the first sample score well and the second drag the retrieval and faithfulness averages down — and, reading those low numbers, you could correctly diagnose sample two as both a retrieval failure and a generation failure.

The `_main_` block first demonstrates the from-scratch metric on an obviously grounded answer (high score) and an obviously hallucinated one ("50 days and a free car" — low score), then runs the full RAGAS evaluation.

BEST PRACTICE

In a real evaluation, you do *not* hand-write the `response` and `retrieved_contexts` as the sample code does for illustration — you generate them by **running your actual RAG system** on each test question. Only the `user_input` and the `reference` (ground truth) are authored by hand. The whole point of evaluation is to score the system's *real* behaviour; fabricating its outputs would measure nothing. Build a function that takes your RAG pipeline and a test question and returns the genuine answer and contexts, and feed those into the dataset.

9.16 Common Mistakes

- **Evaluating by "looks good."** Eyeballing a few answers is not measurement and cannot tell improvement from regression. Use metrics.
- **Treating RAGAS scores as exact truth.** The judge is an LLM; scores carry noise. Use them for comparison and tracking, not as decimal-perfect verdicts.
- **Measuring only the final answer.** A single quality score tells you *that* something is wrong, not *what*. Measure all four metrics to localise the failure.

- **Confusing answer relevancy with correctness.** Relevancy means on-topic and complete, not true. A relevant answer can be wrong; faithfulness checks truthfulness.
- **A weak or unrepresentative test set.** A test set of only easy questions yields flattering, meaningless scores. Include hard, messy, and out-of-scope questions.
- **Changing several variables between runs.** If you change chunk size, embedding model, and `k` together, a score shift cannot be attributed to any of them. Change one thing at a time.
- **Fabricating the system's outputs.** `answer` and `contexts` must come from running the real system, not be hand-written.
- **Skipping a "should refuse" set.** Without out-of-scope questions in the test set, you never measure whether the system hallucinates on the unanswerable ones.

9.17 Best Practices

- **Measure, never eyeball** — replace "looks good" with the four metric scores.
- **Always read all four metrics together** to localise failures to retrieval or generation.
- **Curate a representative, version-controlled test set** with hand-written ground truth, including hard and out-of-scope questions.
- **Generate `answer` and `contexts` by running the real system;** author only `question` and `ground_truth`.
- **Change one variable per evaluation run** so score changes are attributable.
- **Automate evaluation** into a repeatable pipeline and run it after every meaningful change.
- **Use stratified evaluation and ablation** — scores per question type, and with/without each component.
- **Track a baseline and watch for regressions** over time; use RAGAS scores as reliable *relative* signals.

9.18 Real-World Applications

- **Development-time tuning** — measuring whether a change to chunk size, embedding model, retriever, or prompt actually helped.
- **Component justification** — ablation evaluation to prove (or disprove) that re-ranking, RAG Fusion, or a corrective loop earns its added cost.
- **Model and configuration selection** — comparing two embedding models or two LLMs on the same test set with the same metrics.
- **Pre-deployment quality gates** — a minimum metric bar a system must clear before release.

- **Production regression monitoring** — re-running the suite on a schedule to catch silent degradation from data updates or model changes.
- **CI/CD for AI** — wiring `evaluate()` into the deployment pipeline so a change that drops the scores fails the build (Module 10).
- **Stakeholder reporting** — turning "the chatbot seems fine" into defensible numbers for non-technical decision-makers.

9.19 Mini Project: An Evaluation Suite for Your RAG System

Turn evaluation into a habit by building a reusable suite:

1. **Build a test set.** Take one of your RAG systems from earlier modules and write 15–20 test questions with hand-authored ground-truth answers. Deliberately include several hard questions and at least three whose answers are *not* in the knowledge base.
2. **Write a run-and-collect function** that takes your RAG system and a question and returns the genuine generated `answer` and retrieved `contexts`.
3. **Run the full evaluation.** Build the RAGAS dataset by running your system over the whole test set, and score it with all four metrics. Record the baseline numbers.
4. **Diagnose.** Read the four scores. Which is weakest? Using Section 9.9, state whether your system's main problem is retrieval or generation.
5. **Run a controlled experiment.** Make one targeted change addressing the diagnosed weakness (e.g. if context recall is low, increase `k` or change the chunking). Re-run the evaluation. Did the targeted metric improve? Did anything else regress?
6. **Run an ablation.** Add a component from Module 6 (say, re-ranking), evaluate with and without it, and report its measured contribution.
7. **Stratify.** Split your scores by question type — easy, hard, out-of-scope — and report each group separately. Confirm whether the system honestly refuses the out-of-scope questions.
8. **Write up** a short evaluation report: baseline scores, the diagnosis, the experiment result, the ablation result, and a recommendation. This report — measured, reasoned, defensible — is what professional RAG work looks like.

9.20 Chapter Summary

- "Looks good" is not measurement. **RAG evaluation** replaces eyeballing with numbers, by **decomposing** answer quality into distinct measurable dimensions and using an **LLM as judge** to score them. **RAGAS** is the standard framework for this.

- Every RAGAS metric is computed from four **data elements** per test question: `question`, `answer`, `contexts`, and `ground_truth`. The `question` and `ground_truth` are authored by hand; the `answer` and `contexts` come from running the real system.
- The four core metrics split along the two halves of a RAG system. **Retrieval metrics:** *context precision* (are retrieved chunks relevant and well-ranked — signal vs noise) and *context recall* (did retrieval find all needed information — completeness; requires ground truth). **Generation metrics:** *faithfulness* (is the answer grounded in the context — the hallucination measure) and *answer relevancy* (does the answer address the question).
- All four score 0–1, higher is better. A key trap: *answer relevancy is not correctness* — a relevant answer can still be false; faithfulness checks truth against the evidence.
- The decisive value of evaluation is **failure localisation**: low retrieval metrics point to the retriever (Modules 2–6); low generation metrics point to the generator (Modules 1, 7). The four-metric pattern diagnoses *which* component to fix.
- The `evaluate()` **API** runs chosen metrics over an evaluation dataset using the judge LLM. RAGAS's API evolves between versions — the concepts are stable, the import names are not; check current docs.
- An **automated evaluation pipeline** — a fixed test set, a run-and-score function, recorded results, a trigger — turns development into controlled experimentation: change one variable, re-evaluate, keep the change only if scores improved.
- **Benchmarking strategies:** a representative test set, stratified evaluation by question type, component ablation to justify cost, baseline and regression tracking, and a "should refuse" set of out-of-scope questions.
- Use RAGAS scores as reliable **relative** signals for comparison and tracking, not as absolute, decimal-perfect verdicts.

9.21 Practice Exercises

1. **Conceptual.** Explain, in under 120 words, why measuring only the final answer is insufficient for improving a RAG system, and what measuring four dimensions adds.
2. **Metric matching.** For each symptom, name the metric that would be low and whether it diagnoses retrieval or generation: (a) the answer states a fact found nowhere in the retrieved chunks; (b) the retriever returned five chunks, only one relevant; (c) the answer is true but rambles about a side topic; (d) the answer is incomplete because a needed chunk was never retrieved.
3. **Faithfulness vs relevancy.** Construct an example question and answer where answer relevancy is high but faithfulness is low. Explain why this combination is dangerous.
4. **Ground truth.** Which two of the four core metrics typically require a hand-written ground-truth answer, and why can they not be computed without it?

5. **Diagnosis.** A system scores: context recall 0.45, context precision 0.80, faithfulness 0.92, answer relevancy 0.90. Diagnose the system, name the highest-priority fix, and say which earlier module you would revisit.
6. **Code.** Run `module9_evaluation.py`. For Part A, add a third answer that mixes one true and one false claim, and confirm the from-scratch faithfulness score is around 0.5. Explain the result.
7. **Experiment design.** Describe a controlled evaluation experiment to decide whether adding cross-encoder re-ranking (Module 6) is worth its latency cost. State what you hold fixed, what you vary, and which metrics you would read.

9.22 Interview Questions

BEGINNER

Q1. Why is evaluating a RAG system harder than evaluating a classification model?

A RAG answer is open-ended free text — there is no single correct string to compare against, the way there is for a classification label. You cannot just check whether the output equals an expected value, so quality must be decomposed into measurable dimensions and judged.

Q2. What is RAGAS?

An open-source framework for evaluating RAG pipelines. It uses an LLM as a judge to automatically score several distinct quality dimensions — faithfulness, answer relevancy, context precision, and context recall — across a test set.

Q3. What four data elements does RAGAS need per test question?

The question, the generated answer, the retrieved contexts, and the ground-truth (reference) answer. The question and ground truth are authored by hand; the answer and contexts come from running the real RAG system.

Q4. What does faithfulness measure?

Whether the generated answer is factually consistent with the retrieved context — whether every claim in the answer is supported by the context. It is the direct, quantitative measure of hallucination.

INTERMEDIATE**Q5. Explain the four core RAGAS metrics and what each diagnoses.**

Context precision — are the retrieved chunks relevant and well-ranked (signal vs noise); diagnoses retrieval. Context recall — did retrieval find all the information needed (completeness); diagnoses retrieval. Faithfulness — is the answer grounded in the retrieved context (hallucination); diagnoses generation. Answer relevancy — does the answer actually address the question; diagnoses generation. The retrieval pair measures the retriever; the generation pair measures the generator.

Q6. What is the difference between context precision and context recall?

Context precision asks whether what was retrieved is *clean* — relevant and well-ranked, low noise. Context recall asks whether what was retrieved is *complete* — whether the retriever found every chunk needed to answer fully. Precision penalises noise; recall penalises missing information.

Q7. Why is a high answer-relevancy score not the same as a correct answer?

Answer relevancy measures whether the answer is on-topic and addresses the question; it says nothing about whether the facts are true. A confident, on-topic, completely wrong answer can score high on relevancy. Faithfulness is the metric that checks truthfulness against the retrieved evidence — relevancy and faithfulness must be read together.

Q8. How should RAGAS scores be interpreted, given that the judge is an LLM?

As reliable relative signals, not absolute truth. Individual scores carry noise because an LLM produced them, and the same run can vary slightly. They are trustworthy for comparison and tracking — "version B scored higher than version A across the test set" — rather than as decimal-perfect verdicts.

ADVANCED**Q9. How do the four metrics let you localise a failure to a specific component?**

Because the metrics split along the two stages of RAG. If context precision or context recall is low, the retriever is at fault — fix chunking, the embedding model, k , the retrieval technique, or re-ranking. If retrieval metrics are healthy but faithfulness is low, the generator is hallucinating — fix the grounding prompt, temperature, or add a faithfulness-checking pattern. If retrieval and faithfulness are fine but answer relevancy is low, the generator is producing grounded but off-topic answers — fix the prompt. The pattern of four scores points to the exact stage to repair.

Q10. How would you design an automated evaluation pipeline and use it in development?

Build a fixed, version-controlled test set of representative questions with hand-written ground-truth answers, including hard and out-of-scope ones. Write a function that runs the RAG system over the whole test set to collect real answers and contexts, then calls `evaluate()` with the four metrics to produce scores. Log each run's scores with a label for the configuration. Run the pipeline after every meaningful change, changing one variable at a time, and keep a change only if the numbers improved. Optionally wire it into CI so a score regression fails the build.

Q11. What is component ablation in RAG evaluation and why is it valuable?

Ablation means evaluating the system with and without a particular component — for example, with and without cross-encoder re-ranking — on the same test set with the same metrics. The score difference is that component's measured contribution. It is valuable because advanced components add latency and cost; ablation is the rigorous way to decide whether a component is actually earning its cost rather than just feeling sophisticated.

SCENARIO-BASED

Q12. Your RAG system scores high on faithfulness and answer relevancy but low on context recall. Users complain that answers are often incomplete. What is happening, and what do you do?

Low context recall means the retriever is failing to fetch all the chunks needed to answer fully — necessary information exists in the knowledge base but is never retrieved. High faithfulness and relevancy confirm the generator is doing its job well: it produces grounded, on-topic answers from whatever it is given — but it cannot include facts it never received, so answers come out incomplete. The fix is entirely on the retrieval side: revisit chunking strategy and chunk size (a needed fact may be split awkwardly), the embedding model, the value of *k* (too low), and the retrieval technique — consider multi-query retrieval or hybrid search to widen recall. Then re-run the evaluation and confirm context recall has risen.

Q13. A teammate proposes shipping a RAG feature, citing that it answers their ten favourite test questions perfectly. As the reviewer, what concerns do you raise and what do you require before approval?

Ten hand-picked questions are not a basis for a release decision: they are likely the easy, in-scope questions the system was implicitly tuned on, and they reveal nothing about hard, messy, multi-hop, or out-of-scope queries. I would require a proper evaluation: a representative, version-controlled test set of meaningful size, including hard questions and a "should refuse" set of out-of-scope questions; the four RAGAS metrics scored across that set; stratified scores by question type so weaknesses are not hidden by an average; and confirmation that the system honestly declines the unanswerable questions rather than hallucinating. Approval should depend on the system clearing an agreed metric bar on that proper evaluation, not on a flattering handful of demos.

END OF MODULE 9

You can now *prove* a RAG system works — and pinpoint exactly what to fix when it does not. Every component, every advanced pattern, every agentic loop from Modules 1-8 can now be validated with numbers instead of hope. **Module 10 — the Capstone Project** — brings the whole book together: a production-ready RAG system with multi-document ingestion, hybrid retrieval and re-ranking, an agentic LangGraph workflow, a RAGAS evaluation suite, LangSmith monitoring, and full deployment — API, Docker, and cloud.